

Report - Final Exam

ALGORITHMS FOR MASSIVE DATA

Sara Baassou 1003567

A Sketch-Based Approximate SQL Engine for Massive Streaming Data

ABSTRACT

Nowadays, accurately processing queries becomes increasingly complex as datasets grow larger. Storage constraints and ingestion rates make it a particularly challenging task. Therefore, in this project, we propose an approximate SQL engine for massive data streams. And we chose to focus on healthcare since it is a domain where we often face huge dataset. Therefore, we found interesting to implement a system which is capable of processing undesired drug reaction reports in real time without requiring the storage of the all dataset. This was possible thanks to the OpenFDA FAERS (FDA Event Reporting System) dataset. Indeed, the Food and Drug Administration in the USA created this dataset which "collects reports of adverse events and medications errors for drugs and biological products".

1. Introduction
 2. State of the art
 3. Theoretical Analysis
 4. Implementation Details
 5. Experimental Results
 6. Conclusions
 7. Bibliography
-

1. INTRODUCTION

The very fast growth of digital data has made it increasingly difficult to scale traditional data processing systems. In fields such as healthcare, modern

systems must handle massive data streams in real time, all while adhering to strict memory and latency constraints. Indeed, in many cases, storing all the data is prohibitively expensive, or even impossible. Let's consider the example of pharmacovigilance. This field uses very large databases that are constantly updated with new reports of adverse drug reactions. The FAERS (FDA Event Reporting System) in the United States is a prime example. Their publicly accessible database contains millions of reports detailing side effects, the responsible medications, the characteristics of the patients involved, and the medical consequences. The sheer volume of information they contain, the constant addition of new data, and their heterogeneous structure make these datasets extremely complex and large.

This project investigates the following question:

How can we analyze massive healthcare data streams in real time without storing the data explicitly, while using only a very small amount of memory?

To answer this, we created an approximate SQL engine which uses probability to make accurate estimations. Instead of storing the entire dataset, which, as said earlier, would be almost impossible, the system conserves compact summaries of informations, called "sketch", that are updated each time new reports are received via the OpenFDA API. This sketch allows to the engine to answer some analytical queries without having to store all of the data, which reduces drastically memory consumption. This approach allows to process large amount of data without requiring a large storage space. So, the implemented system supports several classes of approximate queries, including: global frequency estimation (`COUNT(*)`), conditional frequency queries (`WHERE`), average aggregation (`AVG`), approximate join-size estimation, approximate `GROUP BY` queries, heavy hitter (`TOP-K`) detection, and approximate distinct counting.

This project uses some key methods that come from how we handle huge amounts of data. Specifically, it uses ideas from two main areas: streaming and sketching algorithms. Indeed, we used the Count-Min Sketch for frequency estimation, and the Tug-of-War sketches for second-moment and join estimation.

The objective of the project was not to build a fully functional DBMS, but rather to demonstrate how modern randomized streaming techniques can emulate important SQL functionalities while drastically reducing memory usage.

Here's how the rest of the report will be laid out. First, we'll look at what have been done in this area. Then we will focus on the theory behind sketch-based data structures. After what, we will talk about the algorithms we have implemented. After that, we will dive into the details of our approximate SQL engine and how it handles streaming data from FAERS. Next, we will share the results of our experiments, where we tested how much memory our system uses, how long it takes to run, how well it scales, and how good its approximations are. Finally, we will sum up what we have found and talk about what could be the next step.

2. STATE OF THE ART

The question around how to analyze massive data streams in real time without storing the data explicitly, while using only a very small amount of memory is a question that is treated and approached by several researcher. indeed, this has led to a rich line of work on streaming algorithms and data sketches, whose goal is to maintain compact summaries of the stream that support approximate queries with provable error bounds. A first family of techniques are frequency and moment sketches. The Count-Min Sketch (Cormode–Muthukrishnan) is a hash-based structure that approximates item frequencies with an additive error of at most $\varepsilon \cdot m$ and failure probability δ , using a $\mathcal{O}(\log(1/\delta)/\varepsilon)$ table. It is widely used in streaming systems for heavy-hitter detection, approximate histograms, and query optimization. More generally, the theory of frequency moments (Alon–Matias–Szegedy) introduced sketches for estimating quantities like $F_2 = \sum_i f_i^2$, which underpin algorithms such as Tug-of-War for second-order frequency moment and join-size estimation.

Another core line of work concerns cardinality and distinct counting.

The Flajolet–Martin algorithm and its successors, notably HyperLogLog, estimate the number of distinct elements in a stream using a few kilobytes of memory, and are now standard in production systems for web analytics and large-scale logging. These sketches are often embedded directly into database engines and big-data platforms to support approximate `COUNT(DISTINCT)` queries.

On the systems side, projects like Apache DataSketches and industrial libraries from Yahoo, Twitter, and others have shown that integrating sketches (Count-Min, HyperLogLog, quantile sketches, etc.) into query engines can reduce query times from hours to seconds while keeping memory usage almost constant. These systems typically expose sketch-aware operators for

cardinality, frequency estimation, quantiles, and set operations, and provide adaptors for SQL engines such as Hive, Pig, PostgreSQL, and BigQuery.

In the specific context of massive healthcare data, most existing work focuses either on batch analytics (e.g., data warehouses over EHRs) or on streaming pipelines that compute fixed aggregates (counts, alerts) rather than general SQL queries. The lecture notes "Algorithms for Massive Data" by Prezza explicitly highlight SQL sketches as a way to bridge this gap: using Tug-of-War, Count-Min, and related sketches to support approximate `COUNT`, `AVG`, `JOIN` size, and frequency queries directly over streams, without storing the raw data.

3. THEORETICAL ANALYSIS

This section presents the theoretical foundations of the probabilistic sketches used in SketchDB. The goal is to justify why these structures allow real-time analytics on massive data streams while using sublinear memory. We analyze the two main components of the system: the Tug-of-War sketch, and the Count-Min sketch. For each, we summarize the estimator, its bias, its variance, and its memory guarantees. When the full proofs are beyond the scope of this report, we cite the classical references in streaming algorithms.

3.1. Streaming Model

SketchDB operates in a streaming model, where data arrive as a sequence of updates : (i, Δ) modifying the frequency of item i by Δ . The algorithm must process each update in $O(1)$ time and maintain a summary using sublinear memory, typically polylogarithmic in the size of the domain. A sketch is therefore a function $[S : \text{Stream} \rightarrow \mathbb{R}^k]$ such that queries (COUNT, SUM, JOIN size, frequencies) can be approximated from S with provable error bounds.

3.2. Tug-of-War Sketch

The Tug-of-War (ToW) sketch, introduced by Alon, Matias and Szegedy (AMS), is a randomized estimator for the second-order frequency moment $F_2 = \sum_i f_i^2$, and for the scalar product between two streams. It relies on a 4-wise independent hash function $h : \mathcal{U} \rightarrow -1, +1$. Given a stream of updates, the sketch maintains $Z = \sum_i h(i)f_i$.

The AMS estimator is $\hat{F}_2 = Z^2$, the unbiasedness is $\mathbb{E}[Z^2] = F_2$, and the variance is $\text{Var}(Z^2) \leq 2F_2^2$. To reduce the variance, the sketch we made sure

that the sketch is repeated t times and the median is returned. We reached a memory usage of $O(t)$, independently of the stream size.

Specifically, we used ToW in our SketchDB to approximate $COUNT(*)$ by applying it to a unique row identifier. We also used it to approximate the JOIN size between two tables A and B using the scalar product: $\langle f_A, f_B \rangle = \sum_i f_A(i) f_B(i)$.

3.3. Count-Min Sketch

The Count-Min Sketch (Cormode & Muthukrishnan, 2005) is a compact structure for estimating item frequencies in a stream. It uses a 2D array of counters of size $t = \lceil \ln(1/\delta) \rceil$, $s = \lceil e/\epsilon \rceil$, with t pairwise-independent hash functions. To estimate an item x , we compute: $\hat{f}(x) = \min_{j=1}^t \text{table}[j][h_j(x)]$. Then we compute the error, with probability at least $1 - \delta$, $f(x) \leq \hat{f}(x) \leq f(x) + \epsilon m$, with m is the total number of updates. This uses $O\left(\frac{1}{\epsilon} \log \frac{1}{\delta}\right)$ counters, independently of the domain size.

In SketchDB, we used Count-Min Sketch to estimate categorical frequencies, frequencies of drug names, reaction, etc., also to support approximate queries like `COUNT(*) WHERE col = value.`, and finally to detect common drugs or reactions in FAERS.

3.4. Flajolet–Martin Sketch

The Flajolet-Martin Sketch, or FM_Sketch, tracks the maximum number of trailing zeros in hashed values $R = \max_x \text{tz}(h(x))$. The estimator is: $\hat{D} = 2^R$. And we used it to estimate `COUNT DISTINCT` queries.

3.5. Combining Sketches for SQL Query Processing

SketchDB integrates these sketches to support a subset of SQL:

SQL Operation	Sketch / Structure Used	Complexity	Error
<code>COUNT(*)</code>	Exact counter	$O(1)$	0 (exact)
<code>SUM(col)</code>	Exact accumulator	$O(1)$	0 (exact)
<code>AVG(col)</code>	Exact SUM / COUNT	$O(1)$	0 (exact)
<code>COUNT WHERE col = value</code>	Count-Min Sketch	$O(1)$	$+\epsilon \cdot m$ (overestimation only)

SQL Operation	Sketch / Structure Used	Complexity	Error
GROUP BY col	Count-Min + observed set	$O(k)$	$+\epsilon \cdot m$ per value
TOP-K(col)	Count-Min + observed set	$O(k \log k)$	Approximate ranking
COUNT DISTINCT(col)	FM Sketch	$O(1)$	High variance ($\approx 50\%$)
JOIN size(col1, col2)	Tug-of-War scalar product (25 reps)	$O(1)$	Unbiased, variance $\downarrow$ with reps

4. IMPLEMENTATION DETAILS

The project SketchDB is implemented in Python and organized around three core components: probabilistic sketches, a sketched table abstraction, and an approximate SQL engine. The system processes FAERS data as a real-time web stream and never stores raw records, in accordance with the streaming model.

4.1 System Architecture

```

OpenFDA Stream
  ↓
Stream Loader
  ↓
SketchDB
  ├── patients
  ├── drugs
  └── reactions
    ↓
MiniSqlEngine
  ↓
Approximate Queries

```

```

file architecture:
engine.py
exact_baseline.py
experiments.ipynb -- file to execute to
see tests/experiments
sketches.py
stream_loader.py

```

4.2. TableSketch

In sketches.py we implemented a TableSketch class which allows to maintain for each column maintains an exact row count, an exact sum for numerical columns, a Count-Min Sketch for frequencies, a Tug-of-War Sketch for join size, a FM Sketch for distinct count, and a set of observed values for GROUP BY and TOP-K. Moreover, the `update_row()` function processes each FAERS record in $O(1)$ time per column.

4.2. MiniSqlEngine

In the file `engine.py`, we implemented a `MiniSqlEngine` classe to be able to support several SQL queries such that:

- `SELECT COUNT(*)`
- `SELECT AVG(col)`
- `SELECT COUNT(*) WHERE col = value`
- `SELECT COUNT(DISTINCT col)`
- `SELECT col, COUNT(*) GROUP BY col`
- `SELECT TOP K`
- Join size estimation via `JOIN` keyword

These queries are parsed using regular expressions and mapped to sketch operations.

4.3. Streaming Loader

Finally, one of the core object of this project is to stream the data. So in the file `stream_loader.py`, we implemented the `OpenFDAStreamLoader` classe which fetches FAERS data from the OpenFDA API. It then, normalizes each record into patients/drugs/reactions and updates the corresponding `TableSketch`. This ensures constant memory usage.

5. EXPERIMENTAL RESULTS

Finally, we arrive at the experimental results section. Here, we will evaluate the performance of SketchDB according to three dimensions: streaming speed, memory footprint and approximation accuracy. All experiments were conducted using the FAERS (FDA Adverse Event Reporting System) dataset distributed directly from the OpenFDA API, without storing raw data locally. The results come from the “`experiments.ipynb`” execution. The output values differ from an execution to another but not in a relevant way. for further informations (graphs, values, etc.) please see the file.

5.1. Experimental Setup

First thing, let’s recall the context. The experiments were conducted on a standard laptop using Python 3.x, no GPU, and 8–16 GB RAM. The streamed dataset is the FAERS drug event reports streamed via OpenFDA. We evaluated

the system on streams of 100, 500, 1000, 2000, and 5000 records. Also, we used the following sketch parameters:

- Count-Min Sketch: $\epsilon = 0.01$, $\delta = 0.01$
- Tug-of-War: 1 repetition (default), seeded per column

And we measured the following metrics:

- streaming throughput (records/s),
- memory usage (kB),
- approximation error,
- and scalability with increasing stream sizes.

5.2. Treaming Throughput

One of the main objective of this project was to process massive data in short execution time. The following table shows that this goal was achieved. Indeed, it shows that the treaming throughput remain fairly constant as the size of processing streams grows.

Stream Size	time	throughput
100	4.376822	22.847628
500	24.772927	20.183323
1000	43.792749	22.834831
5000	95.918414	20.851054

5.3. Memory Usage

One other main objective was to process massive streams while using very little memory. So to evaluate wether this objective was achieve or not we measured the RAM used after processing streams of different sizes.

We used `psutil` inside the notebook, and we observed:

Stream Size	RAM Used by SketchDB
100	5.171875
500	3.156250
1000	12.453125
2000	4.062500

Stream Size	RAM Used by SketchDB
5000	23.515625

The memory footprint vary a little but compared to storing the complete dataset, this remains extremely small.

5.4. Approximation error analysis

To evaluate the approximation quality, we compared sketch-based estimates against exact computations obtained on a small validation subset temporarily stored in memory. We did so for the frequency estimation, the distinct counting estimation, and the join size estimation.

Query	Exact Value	Approximation	Relative Error
count patient	2000.0000	2000.0000	0.0
avg age	35.4365	35.4365	0.0
count where	14.0000	16.0000	0.142857
count(distinct)	0	3.0000	3.000000
count distinct 2	1659.0000	512.0000	0.691380

5.5. Other observations

To go further, we tested the requests for knuckle sizes, heavy typing, and grouping by. All these requests were processed in a very short time and with little storage.

6. CONCLUSIONS

In this project, we presented **SketchDB**, an approximate SQL engine. The system was built around probabilistic sketching techniques and evaluated on real-world reports. The main objective was to answer the following question:

How can we analyze massive healthcare data streams in real time without storing the data explicitly, while using only a very small amount of memory?

To answer this question, we implemented a streaming architecture based on compact probabilistic summaries. Instead of storing all incoming records, SketchDB continuously updates sketches. This allows the system to respond to SQL-like analytical queries while using little memory. To do so, the project integrates the following classical streaming techniques, the Count-Min

Sketch for approximate frequency estimation, the Tug-of-War sketches for join-size estimation and frequency moments, and the Flajolet-Martin sketches for approximate distinct counting. Using these structures, SketchDB supports the following approximate SQL operations : Count, Count Where, avg, Group By, Top-K, Count Distinct, and Join. The experimental evaluation showed that the system successfully achieves the goals of streaming analytics. Indeed, it achieved constant memory usage despite increasing stream sizes. It also achieved fast streaming throughput, accurate frequency estimations for common analytical queries, and the ability to process real healthcare data in real time.

Overall, the project demonstrates that probabilistic data structures can replicate important database functionalities while requiring only a fraction of the memory required by traditional systems. Although SketchDB is not intended to replace a complete DBMS, it illustrates how modern streaming algorithms can provide scalable and practical solutions for massive data analysis.

Several improvements could be explored in future work. We could think of increasing the number of sketch repetitions to reduce variance, or just supporting more advanced SQL operators. We can also think of replacing Python with a lower-level implementation (e.g. C++ or Rust) for higher throughput, and evaluating the system on significantly larger real-world streams.

7. BIBLIOGRAPHY

1. Alon, Matias, Szegedy — *The Space Complexity of Approximating the Frequency Moments* (1996).
2. Cormode, Muthukrishnan — *An Improved Data Stream Summary: The Count-Min Sketch and its Applications*(2005)
3. Flajolet, Martin — *Probabilistic Counting Algorithms for Data Base Applications* (1985).
4. Muthukrishnan — *Data Streams: Algorithms and Applications* (2005)
5. OpenFDA Documentation
6. Lecture notes: *Algorithms for Massive Data*, Nicola Prezza, Ca' Foscari University of Venice.